A Learned Pruning Budget for SED (Category Early Rejection)

Assignment 4 Iteration 1

Code uploaded to: Github

Shashank Sadashiv, Arunabh Singh, Maniar Sheil Dipenkumar, and Samarth Bhatnagar

The learned k predictor is a small neural network that decides how many candidate classes SED should consider at each location in an image. It looks at the scores SED already produces near the start of its decoder, chooses a budget for that image, and passes the budget to SED's Category Early Rejection mechanism. The budget controls how aggressively the decoder prunes class candidates. SED still produces the final pixel labels.

For example, a photograph of a largely empty road may produce strong scores for road, sky and vegetation across most of the frame. A street market can contain several smaller objects, surfaces and people whose scores overlap. A small budget can save decoder work in the first scene. The second scene may need more candidate classes to protect a category that receives a weak early score. The predictor tries to recognize this difference before SED spends computation at higher resolutions.

How this builds on SED

Semantic segmentation assigns a category to each pixel, so an output can distinguish the road from the sidewalk even when both cover large regions. SED is open vocabulary because it compares image features with embeddings of class names supplied as text. The benchmark still gives it a finite list of candidate names for each run. On ADE20K-150, SED must choose among 150 names such as wall, road, chair and cabinet. [2]

SED's original rejection rule already reduces computation. It applies the same per-location setting, k=8, across images. Our change is to predict that setting from the image and reuse the chosen value at all three rejection stages. We kept the released image encoder, text encoder and segmentation decoder frozen so the experiment could isolate the effect of budget selection.

Research question

We asked whether an image-specific k can make the frozen model faster than fixed k=8 while preserving segmentation accuracy. We treated the fixed settings, the full decoder, and a rule based on confidence and entropy as separate controls. The expected improvement was a better accuracy and latency tradeoff. The prediction head can only change which classes survive and interact in the decoder. Its gains therefore depend on whether the coarse scores contain enough information to choose a useful budget.

What the pruning budget controls

From the image to a coarse class score

The ConvNeXt-B image encoder receives an internally resized image of 768 × 768 pixels. Its coarsest features lie on a 24 × 24 grid, with one feature vector for each of the 576 spatial locations. The text encoder supplies vectors for the candidate class names in several prompt templates. SED compares normalized image and text vectors with cosine similarity. The resulting cost volume records how well each location matches each class under each prompt. Higher similarity means a stronger match. [2, 3]

SED embeds those similarities and uses its first auxiliary scoring head, head0, to produce one score per class per location. With the 150 ADE20K class names, the tensor presented to the new policy has shape 1 × 150 × 24 × 24. The leading 1 is the batch size. The remaining axes index classes, grid rows and grid columns. SED also projects features from its encoder to guide later decoder stages as they reconstruct the segmentation at finer scales. [2, 3]

The top k rule and the union across locations

The number k specifies how many class scores each location may nominate. SED ranks the scores at every location, takes the top k names, and builds the union of all names nominated anywhere in the image. The entire class feature map survives if any location nominated it. SED applies this rule before each of its three upsampling stages. Once a class has been dropped, later stages cannot recover its feature map. [2, 3]

Example location	First choice	Second choice	Third choice
A	Wall	Floor	Table
B	Floor	Chair	Wall
C	Table	Wall	Cabinet

In this example, k=1 keeps wall, floor and table because those names win somewhere. At k=2, the union also includes chair. Cabinet survives only at k=3. This example has three locations. On a 24 × 24 grid, k=4 can keep dozens of classes if different locations nominate different names. Two images decoded with fixed k=8 can therefore retain different numbers of classes.

Larger values of k tend to preserve more plausible alternatives and increase the work in the later decoder stages. Smaller values reduce work but can remove a valid class before its boundaries are refined. We also measured the dense decoder, which carries all classes through every stage. The encoder and initial cost-volume work occur before the budget decision in every mode, so only the later portion of inference can benefit from pruning.

Why we selected the learned predictor

Task 3 produced five ideas for the problem of adaptive Category Early Rejection and another five for the problem of near-synonym confusion. We used the original novelty, feasibility, clarity and impact scores. The learned k predictor earned 18 out of 20, the highest total across both sets. The nearest idea from the other problem earned 17 and would change SED's fine-tuning loss. [1]

Rank	Task 3 idea	N / F / C / I	Total	Reason for the comparison
1	Learned k predictor	4 / 4 / 5 / 5	18	We can generate a target for each image by measuring what happens at several fixed budgets.
2	Confidence and entropy rule	3 / 5 / 5 / 3	16	It adapts to the image without fitting a model. We used it as the main adaptive baseline.
3	Two-pass adaptive rejection	3 / 5 / 4 / 3	15	Its first pass would add decoder work to every prediction, including simple scenes.
4	Rejection by semantic cluster	4 / 3 / 3 / 4	14	Its effect depends on cluster definitions and can keep classes that are absent from the scene.
5	Reinforcement learning policy	5 / 2 / 3 / 4	14	Learning per-stage actions would require many decoder runs and a reward for latency and accuracy.

N = novelty, F = feasibility, C = clarity, I = impact. Each original Task 3 score is out of five.

Why the choice was practical

We could evaluate the idea with the authors' released checkpoint. The frozen model supplied the coarse scores and predicted masks, and only a small classifier needed to learn. This fit the Victus hardware and gave us a direct experiment. We could ask whether the head's choices outperform fixed k=4 and k=8 while holding the images, checkpoint and precision constant.

The proposal also gives a way to diagnose a weak result. If the best budget varies across images but the learned head mostly chooses one value, the decision rule has failed to use that variation. If nearly every image favors the same fixed budget, a learned policy has little opportunity to improve the tradeoff.

How the predictor learns

We reduced the first score tensor to ten statistics before training the head. These statistics describe the distribution of scores and winners across the 576 locations. We summarized scores across classes so the same features could describe the COCO-Stuff and ADE20K vocabularies.

Feature group	What we measured and why
Confidence	Mean and standard deviation of the highest sigmoid score at each location. The sigmoid puts each raw score on a 0 to 1 scale. These summaries describe how strong and how variable the winners are.
Separation	Mean and standard deviation of the gap between the top two sigmoid scores, plus the fraction of locations with a gap below 0.05. Small gaps indicate that two classes often compete for the same area.
Uncertainty	Mean entropy of the softmax distribution across classes, divided by the logarithm of the vocabulary size. Values closer to one mean the score mass is spread across more candidates.
Diversity	The logarithm of one plus the number of distinct winners, the most frequent winner's share of locations, and the logarithm of one plus the effective number of winners. That number is higher when the winners are spread evenly across the grid.
Vocabulary size	The logarithm of one plus the number of candidate classes. This input was constant during COCO-Stuff fitting, so the fitted head ignored it.

How we generated the training target

For each labeled COCO-Stuff image, we ran the same checkpoint at k=4, 8, 16, 32 and with all classes. We computed image IoU for each output and assigned the smallest k whose score was at most 0.5 percentage points below the dense score. If no pruned setting qualified, the target was the dense route. We use the word oracle for this target because it requires the ground-truth mask and five evaluated alternatives.

For example, if dense scored 55.0% on an image, a budget had to score at least 54.5%. If k=4 scored 53.4% and k=8 scored 54.7%, the training label was k=8. A pruned setting that exceeded the dense score also qualified. The resulting label is a category for the head to predict, rather than a continuous measure of compute.

How we fitted and used the head

We centered and scaled features using the fitting split, then trained a multilayer perceptron with ten inputs, 32 hidden units, ReLU and five outputs. Its two linear layers contain 517 trainable weights and biases in total. We used cross-entropy and AdamW for 80 epochs in CPU batches of 32, with learning rate and weight decay both 0.001. We kept the epoch with the lowest development loss. During inference, the highest output score chooses a budget without access to a mask.

Repository and model checkpoints

We organized the project as a Git repository named SED Adaptive CER. The source layout stays stable as the experiments change. A commit identifies one iteration of the method, report and compact results. We keep the upstream SED source pinned and apply a small patch so we can see exactly where the new policy enters the original decoder.

Repository path	Role in the experiment
src/sed_adaptive_cer	Contains the budget features and head, the archive readers, the SED inference wrapper, and the evaluation program.
patches/adaptive_cer.patch	Adds the optional budget decision immediately after head0. It also exposes the prepared decoder inputs so multiple budget settings can share the same initial computation during accuracy evaluation.
scripts and tests	PowerShell scripts set up the Windows environment and start the benchmark. The tests cover oracle labels, feature invariance and pixel-level IoU.
submission_docs and results	Store the assignment PDFs, this report, the benchmark summary, per-class IoU, the trained head and the saved split IDs.
Local runtime and runs	Contain source checkouts, checkpoints and ZIP archives outside Git, along with per-image evaluation and timing records. JSON, JSONL and CSV files indexed by image ID serve as the experiment store.

What a checkpoint contains

A checkpoint is a saved set of model parameters. SED publishes checkpoints for the ConvNeXt-B and ConvNeXt-L variants. We used the Base checkpoint because the smaller backbone fit the laptop GPU and corresponds directly to the Base rows in the paper. The checkpoint contains the learned segmentation model, including the fine-tuned image encoder and decoder weights. Dense, fixed and adaptive inference use the same weights. The difference is which classes pass through each stage. [2, 3]

We also used the original LAION OpenCLIP ConvNeXt-B initialization. SED constructs the text prompt embeddings when it builds the model, then loads the released SED weights. The two downloads therefore serve different purposes. The OpenCLIP file provides the initialization used to reproduce the class text features. The SED checkpoint restores the trained segmentation model. The small head that we fitted is saved separately as head.json with its feature normalization statistics. [3]

Software used in the run

We used Python 3.13, PyTorch 2.6 with CUDA 12.6, and torchvision 0.21. Detectron2 built SED and handled resizing and postprocessing. OpenCLIP handled text and timm provided ConvNeXt. einops rearranged tensors, Pillow opened JPEGs and indexed PNGs, and NumPy accumulated pixel counts and computed percentiles.

Datasets and GPU compute

Dataset	Size and use in the experiment
COCO-Stuff val2017	5,000 public validation images with dense masks over 171 object and background categories, plus void. We used 640 images to fit the head and another 160 to select its checkpoint. Eight other images were reserved for parity checks and warm-up. We used the label map supplied by SED.
ADE20K A-150	The suffix 150 is the size of the evaluation vocabulary. The scene-parsing release contains 20,210 training images and 2,000 labeled validation images. We scored all 2,000 validation images. A deterministic subset of 256 validation images supplied the timing measurements.

SED's released checkpoint was trained on COCO-Stuff. Fitting our head on that source domain let us transfer the full system to ADE20K without fitting or selecting model parameters on ADE20K labels. For A-150, we supplied SED with the 150 ADE20K class names in the released order. We converted mask IDs 1 through 150 to class indices 0 through 149. The raw void value 0 became ignore index 255 and contributed no pixels to the accuracy scores. [2–5]

How we used the GPU

We ran the encoder and decoder on an NVIDIA GeForce RTX 4050 Laptop GPU with about 6 GB of VRAM. Each forward processed one image at SED's 768 × 768 internal resolution. We used FP16 autocast for the frozen GPU model and computed the feature statistics and decision head in FP32. The GPU generated the masks and training targets. The head itself fitted on the CPU because 517 parameters and 800 feature rows are small.

During accuracy evaluation, we computed the image features, cost volume and decoder guidance once per image, then reused those tensors for the five fixed settings. A learned or heuristic decision can select only one of those five settings. We therefore used the corresponding cached mask for accuracy. We checked the masks and retained classes against separate forwards on COCO-Stuff and ADE20K images. For very large masks, we resized groups of class scores and then took the same global winning class per pixel.

The dense decoder reached 2.61 GiB of active PyTorch allocation in the timed pass. That figure measures memory allocated to tensors. GPU-wide use also includes the allocator's reserve and other processes. We kept the data in ZIP archives and read pairs by ID. The five downloaded data and checkpoint files occupied about 3.9 GB on disk. The setup script creates a separate local runtime for Python and the pinned source trees.

Evaluation and baseline definitions

Accuracy across the full validation split

We evaluated the seven policies on the same 2,000 images with the same checkpoint, class names, input size, resizing and precision. For each class, intersection over union (IoU) divides the number of correctly predicted pixels by the number that either the model or the annotation assigns to that class. For example, 60 correctly predicted chair pixels, 20 false chair predictions and 20 missed chair pixels give an IoU of 60 divided by 100, or 60%.

For dataset mIoU, we first added pixel counts for each class across all validation images and then averaged the resulting class IoUs. This gives each annotated class equal weight in the final score. The image IoU used for an oracle label instead averages over the classes annotated in that one image. We also reported mIoU for crowded scenes, defined in advance as images with at least ten annotated classes. Class retention measures the share of annotated image and class pairs whose class survived the final rejection stage.

What the seven policies do

We used dense SED and fixed k=4, 8, 16 and 32. Fixed k=8 was the primary reference because it is the setting used in the original CER comparison. We also evaluated our learned head and a deterministic confidence and entropy heuristic. For the heuristic, we counted how many different classes ranked first somewhere on the coarse grid. We then added eight times the mean normalized entropy, rounding that addition upward. We clipped the sum to the range from 4 to 32 and rounded up to an available fixed budget.

For example, six different winning classes and mean normalized entropy of 0.50 give six plus four, or ten. The heuristic chooses k=16, the smallest available budget at least as large as ten. The additional allowance responds to uncertain class scores. The learned head uses more features and the measured oracle labels to make its choice.

Latency and acceptance criteria

We warmed up each method on eight separate images and then timed all methods independently on 256 seeded validation images. We varied the order of the seven methods per image. A timed call included copying the prepared image tensor to the GPU, the full model, output resizing and choosing the final class for each pixel. We synchronized CUDA so the recorded wall time included completed GPU work. Image loading and CPU preprocessing were outside the timer.

The p50 is the median latency. The p95 is the 95th percentile, so roughly 5% of measured calls took longer. We compared both because a change can speed up typical images while slowing down difficult ones. The target relative to fixed k=8 was at most 0.5 percentage points of mIoU loss, lower p50, and at most a 10% increase in p95.

Full validation results

We evaluated all 2,000 validation images and observed annotations for 150 of the 150 classes. There were 696 images with at least ten annotated classes. Each accuracy figure below uses the full validation set or the stated crowded subset. The latency and peak allocation columns come from the 256 independent timed images.

Policy	mIoU %	Crowded mIoU %	p50 ms	p95 ms	Median classes kept	Peak GiB
Dense SED	31.74	32.18	456.7	501.6	150	2.61
Fixed k=4	31.35	31.47	174.4	228.6	27	1.60
Fixed k=8	31.62	31.81	214.4	277.4	43	1.87
Fixed k=16	31.74	32.02	281.5	344.6	68	2.20
Fixed k=32	31.68	32.03	371.7	424.2	105	2.62
Entropy heuristic	31.71	32.04	366.2	424.3	102	2.62
Learned k	31.41	31.65	194.3	471.9	33	2.61

The learned policy stayed within the accuracy allowance, losing 0.22 percentage points of mIoU against fixed k=8. Its median latency fell 9.4%. Its p95 rose 70.1%, exceeding the 10% allowance. Crowded-scene mIoU changed by -0.16 points, and annotated-class retention fell from 92.6% to 90.5%.

The head routed 62 of 256 timed images to the dense decoder. Those calls had a median of 458.2 ms and drove the slow tail. Against fixed k=4, the head gained just 0.06 mIoU points and added 19.9 ms at the median.

Which budgets were used

Decision source	k=4	k=8	k=16	k=32	All
COCO fit oracle	278	46	45	70	201
ADE validation oracle	1,035	181	212	224	348
ADE entropy heuristic	0	47	537	1,416	0
ADE learned head	1,517	0	0	0	483

The learned policy chose k=4 for 1,517 images (75.8%). A hindsight policy using the oracle choices scored 32.14% mIoU. Because those choices use the annotations, this result estimates the opportunity under our labeling rule.

Comparison with published results

We compared full A-150 accuracy with the SED paper and later open-vocabulary segmentation papers. The closest comparison is SED's ConvNeXt-B model in Table 7, which evaluates several pruning budgets with the same backbone. We also included two direct successors to SED and a more recent cost-aggregation method. The later systems change how segmentation scores are produced, so they help place the result in the wider literature. [2, 6–8]

Published method	Model or setting	A-150 mIoU	Reported latency
SED, CVPR 2024 [2]	ConvNeXt-B, fixed k=4	31.4%	67.5 ms, A6000
SED, CVPR 2024 [2]	ConvNeXt-B, fixed k=8	31.6%	82.4 ms, A6000
SED, CVPR 2024 [2]	ConvNeXt-B, dense	31.8%	177.7 ms, A6000
DPSeg, CVPR 2025 [6]	ConvNeXt-B, Inference II	33.3%	—
SED++, TPAMI 2026 [7]	ConvNeXt-B	34.9%	69 ms, A6000
PCA-Seg, CVPR 2026 [8]	ViT-B/16	38.1%	—

An empty latency comparison is shown with a dash. These figures were reported by the respective authors.

What the paper comparisons tell us

Our fixed k=8 result was 31.62%, a difference of +0.02 percentage points from SED Table 7. Our learned result was 31.41%, or -1.89 points relative to DPSeg and -3.49 relative to SED++. DPSeg adds visual prompt information to the cost volume. SED++ uses additional rejection in its encoder and additional text embeddings. PCA-Seg uses a different backbone and a different approach to cost aggregation. [6–8]

We used the same published A-150 split and kept head supervision in COCO-Stuff. Even with that alignment, software versions and precision can shift the reproduced SED score. The paper's A6000 timings also come from a different device. We assessed latency gains using our own matched RTX 4050 measurements, which include the predictor and the rest of the inference path.

SED Table 7 shows that fixed k=8 is already within 0.4 percentage points of dense across its five datasets. On A-150, changing k=8 to k=4 saved 18.1% of the reported time while losing 0.2 points of mIoU. The headline 4.7× speedup refers to PC-459. The repository's SED-fast label denotes the CER path. The faster SED variant in the paper's Table 2 also reduces input resolution. Our experiments used one internal resolution across policies. [2, 3]

Shortcomings and future iterations

What we observed

The development head matched the oracle label on 40.6% of images. Always predicting the most frequent fit label would have matched 44.4%. On ADE20K, the learned policy lost more than 0.5 image-IoU points against dense on 729 of 2,000 images. The head chose dense for 88 of the 348 images whose oracle required it. It also chose dense for 247 images where k=4 had already met the oracle tolerance.

The head retained fewer annotated classes than fixed k=8 on 507 images. In ADE_val_00001202, it retained 2 annotated classes and scored 5.18% image IoU. Fixed k=8 retained 3 and scored 29.20%. The comparison shows how a useful candidate can disappear during pruning.

Per-class IoU also varied. Among classes with at least 50,000 annotated pixels, ottoman had the largest drop against fixed k=8 (-10.67 percentage points). The full per-class CSV records which categories were affected.

What limited the current design

A single oracle label tells the head which action first met the tolerance. It hides how far each of the other budgets was from the target and treats a slight mistake like a large one during cross-entropy training. The input also summarizes an entire image. A small region with a rare class can contribute little to those averages. Our COCO-to-ADE transfer tests a second domain, but score distributions can still change with a wider vocabulary or a different dataset.

The five-budget calibration sweep took ~17 minutes of per-image work, while training the head took 6.8 seconds. We measured 45 minutes for full validation and 9 minutes for the timing pass, excluding setup and warm-up. The main compute cost was generating segmentation masks. At inference the policy also incurs score reductions and a GPU-to-host scalar read before later decoder stages can start.

Feedback for the next LLM coding pass

We would first model the accuracy loss from choosing k=4 instead of k=8. The saved per-budget IoUs already contain that supervision. A risk threshold chosen on COCO-Stuff development images could route uncertain cases to k=8 and avoid the dense fallback that drove p95. We would test that single change against both fixed settings on the same protocol and report class retention and tail latency. A separate iteration could test features focused on small, uncertain regions or evaluate transfer on PC-459.

Codex implemented and checked the runner and this report. We adapted the staged validation and run records from the assignment's LLM research references. [9–12]

References and reproducibility records

Prior work by Group 6

[1] Group 6. Research Problem Discovery and Ideation, Task 3, pp. 2 and 4–5. See also the limitation report, L4, for the source research gap.

Segmentation and datasets

[2] Xie et al. SED. CVPR 2024, pp. 3426–3436. Section 3.3 and Tables 2 and 7. CVPR paper

[3] Xie et al. SED reference implementation, evaluation scripts and published checkpoints. Commit db20d93c. Pinned SED source

[4] Caesar, Uijlings and Ferrari. COCO-Stuff. CVPR 2018 and the official COCO 2017 split documentation. COCO-Stuff project

[5] Zhou et al. Scene Parsing through ADE20K. CVPR 2017 and the scene-parsing release. ADE20K benchmark

[6] Zhao et al. DPSeg. CVPR 2025, Table 1, ConvNeXt-B Inference II. CVPR paper

[7] Xie et al. SED++. IEEE TPAMI, 48(3), 2528–2543, 2026. IEEE record

[8] Yin et al. PCA-Seg. CVPR 2026, Table 2, ViT-B/16. CVPR paper

Research workflow references

[9] Li et al. AutoSOTA. Baseline reproduction and experiment evaluation. arXiv 2604.05550

[10] Jin et al. Arbor. Connecting research hypotheses with experimental evidence. arXiv 2606.11926

[11] Weng et al. DeepScientist. Staged validation of research ideas. arXiv 2509.26603

[12] Lyu et al. EvoScientist. Retaining experiment lessons across iterations. arXiv 2603.08127

Run files

We saved the dataset and code hashes, exact split IDs, fitted head, benchmark summary and per-class IoU in results/ade150. The local runs/ade150 directory holds the per-image measurements used to compute this report. The repository README gives the PowerShell commands to reproduce the run.